

TechMecha Torque OS — Enterprise System Blueprint

Workforce Operations + Client Management Platform

Version: 1.0.0 · **Authored by:** Senior Platform Architect · **Classification:** Internal Engineering Spec

Table of Contents

1. [Executive Summary](#)
 2. [Product Philosophy](#)
 3. [Architecture Overview](#)
 4. [Tech Stack — Final Decisions](#)
 5. [Design System](#)
 6. [Module 1 — IMS: Intern Management System](#)
 7. [Module 2 — CMM: Client Management Module](#)
 8. [Shared Platform Features](#)
 9. [Database Schema — Final Architecture](#)
 10. [API Design Standards](#)
 11. [Security & Compliance](#)
 12. [Role-Based Access Control](#)
 13. [Automation Engine](#)
 14. [AI Integration Layer](#)
 15. [Infrastructure & DevOps](#)
 16. [Build Phases & Milestones](#)
 17. [Feature Audit — Removed, Retained, Upgraded](#)
 18. [Success Metrics & KPIs](#)
-

1. Executive Summary

TechMecha Torque OS is an enterprise-grade, unified workforce and business operations platform. It consolidates intern lifecycle management and client relationship management into a single cohesive SaaS product — eliminating the need for separate tools like Zoho HR, HubSpot, and ClickUp for a company of this type.

The platform is designed for:

- **Scale** — Supports 10 to 10,000 users without architectural changes
- **Speed** — Sub-200ms API responses, real-time sync across clients

- **Extensibility** — Modular monorepo design, each domain is independently deployable
- **Security** — SOC 2 Type II aligned, RBAC-enforced, full audit trail

This document is the engineering source of truth. All design, backend, and frontend decisions reference this spec.

2. Product Philosophy

Core Principle: Opinionated, Not Overwhelming

Most enterprise platforms fail because they try to do everything with equal weight. TechMecha Torque OS follows a strict **80/20 principle** — 80% of daily workflows should require no more than 3 clicks. The remaining 20% (edge cases, advanced config) lives in settings and admin panels.

What This Product Is NOT

- Not a general project management tool — it is workforce and client-specific
- Not a replacement for Figma, Jira, or Slack — it integrates with them
- Not a CRM for sales teams — it is for managing active business relationships and tracked intern talent

Design Philosophy

┆ "The best enterprise software feels like a product, not a portal."

Every screen must earn its existence. If a feature cannot be justified by a real daily workflow, it is deferred or removed.

3. Architecture Overview

System Type

Modular Monolith → Microservices-Ready

We begin with a well-structured modular monolith (faster to build, easier to debug) with clean domain boundaries so individual modules can be extracted into microservices as traffic demands.

TechMecha Torque OS

```
|
├─ apps/
│   ├── web/           → Next.js 14 App Router (primary frontend)
│   ├── mobile/        → React Native (Phase 3)
│   └─ api/            → NestJS monolith (domain-separated)
│
├─ packages/
│   ├── ui/            → Shared component library
│   ├── config/        → ESLint, TypeScript, Tailwind config
│   ├── types/         → Shared TypeScript types/contracts
│   └─ utils/          → Shared utility functions
│
└─ infrastructure/
    ├── docker/
    ├── nginx/
    └─ terraform/      → AWS/GCP provisioning
```

Domain Separation (Backend)

```
api/src/
├─ auth/              → Authentication, sessions, RBAC
├─ ims/               → Intern Management (self-contained domain)
│   ├── interns/
│   ├── attendance/
│   ├── tasks/
│   ├── learning/
│   └─ performance/
├─ cmm/               → Client Management (self-contained domain)
│   ├── clients/
│   ├── pipeline/
│   ├── activities/
│   └─ invoices/
├─ shared/            → Notifications, uploads, analytics, AI
│   ├── notifications/
│   ├── storage/
│   ├── analytics/
│   └─ ai/
└─ automation/        → Rule engine, scheduled jobs
```

Data Flow

Browser/App



```
Next.js (SSR + Client)
├── Server Actions → NestJS REST API
├── WebSocket → Real-time Gateway (Socket.io)
│
│   ├── PostgreSQL (primary)
│   ├── Redis (cache + pub/sub)
│   ├── S3-compatible (file storage)
│   └── Vector DB (AI embeddings)
```

4. Tech Stack — Final Decisions

Frontend

Layer	Technology	Rationale
Framework	Next.js 14 (App Router)	SSR, streaming, server actions, SEO-ready
Language	TypeScript (strict mode)	Type safety across monorepo
Styling	Tailwind CSS v4 + CSS Variables	Utility-first, theme-aware
Components	shadcn/ui (customized)	Headless, composable, accessible
Animation	Framer Motion	Gesture + layout animations
State (global)	Zustand	Lightweight, devtools-friendly
Server state	TanStack Query v5	Cache, sync, optimistic updates
Forms	React Hook Form + Zod	Type-safe validation
Charts	Recharts + D3 (custom)	Flexible, composable
Tables	TanStack Table v8	Virtualized, sortable, filterable
Rich text	Tiptap	ProseMirror-based, extensible
DnD	@dnd-kit	Accessible drag and drop
Date	date-fns	Tree-shakable, no moment.js

Backend

Layer	Technology	Rationale
Runtime	Node.js 20 LTS	Stable, ecosystem depth
Framework	NestJS	Decorators, DI, module system
API style	REST (primary) + WebSocket	REST is predictable; WS for real-time
ORM	Prisma	Type-safe queries, migrations, studio
Validation	class-validator + Zod	Runtime type safety on DTOs
Jobs	BullMQ + Redis	Reliable job queues
Search	PostgreSQL Full Text + pgvector	No separate Elasticsearch in Phase 1
Email	Resend (or Postmark)	Transactional email, high deliverability
PDF	Puppeteer (server-side)	Templated, pixel-perfect documents

Database & Infrastructure

Layer	Technology	Rationale
Primary DB	PostgreSQL 16	ACID, JSON support, mature
Cache	Redis 7	Session, pub/sub, rate limiting
File Storage	AWS S3 / Supabase Storage	Scalable, CDN-friendly
Auth	NextAuth.js v5 + JWT	Session management, OAuth
Real-time	Socket.io (adapter: Redis)	Multi-instance WebSocket support
Hosting	Railway / Render (Phase 1), AWS ECS (Phase 2)	Cost-effective to enterprise-ready
CDN	Cloudflare	Static assets, edge caching
Monitoring	Sentry + PostHog	Error tracking + product analytics

Dropped from Original Spec

- **Internet Identity** — replaced with standard NextAuth (Google OAuth + email/password). Internet Identity is Dfinity/ICP-specific and adds unnecessary complexity.
- **Firebase Storage** — standardized on S3-compatible storage via Supabase Storage or AWS S3.

- **Supabase as backend** — used only for storage. Full NestJS backend for business logic control.
 - **WhatsApp API** — replaced with WhatsApp deep linking (wa.me) in Phase 1. Native API requires Meta Business verification (weeks of delay). Full Twilio WhatsApp integration in Phase 2.
-

5. Design System

Theme Tokens

CSS

```
:root {
  /* Core palette */
  --color-bg-base:      #0A0A0A;    /* Primary background */
  --color-bg-surface:    #111111;    /* Cards, panels */
  --color-bg-elevated:    #1A1A1A;    /* Modals, popovers */
  --color-bg-muted:      #1F1F1F;    /* Hover states */

  /* TechMecha Red */
  --color-accent:        #E8192C;    /* Primary actions */
  --color-accent-muted:  #E8192C1A;  /* Ghost backgrounds */
  --color-accent-hover:  #FF2D3D;    /* Hover */

  /* Text */
  --color-text-primary:   #F0F0F0;
  --color-text-secondary: #888888;
  --color-text-muted:     #555555;
  --color-text-inverse:   #0A0A0A;

  /* Borders */
  --color-border:         #2A2A2A;
  --color-border-focus:   #E8192C66;

  /* Status */
  --color-success:        #22C55E;
  --color-warning:        #F59E0B;
  --color-error:          #EF4444;
  --color-info:           #3B82F6;

  /* Typography */
  --font-display:         'Geist', sans-serif;    /* Headings */
  --font-body:            'Geist Mono', monospace; /* Body, data */

  /* Radii */
  --radius-sm:  4px;
  --radius-md:  8px;
  --radius-lg:  12px;
  --radius-xl:  16px;

  /* Shadows */
  --shadow-sm:  0 1px 3px rgba(0,0,0,0.4);
  --shadow-md:  0 4px 20px rgba(0,0,0,0.5);
  --shadow-glow: 0 0 20px rgba(232,25,44,0.15);

  /* Glassmorphism */
  --glass-bg:    rgba(17,17,17,0.8);
  --glass-border: rgba(255,255,255,0.06);
```

```
--glass-blur:    blur(16px);  
}
```

Core Component Library

Every component must be built and documented as a standalone `packages/ui` export.

Layout Components

- `<Shell>` — App shell with sidebar, topbar, and content area
- `<Sidebar>` — Collapsible, keyboard navigable, persisted state
- `<PageHeader>` — Title, breadcrumbs, actions slot
- `<ContentArea>` — Scroll container with safe zones

Data Display

- `<DataTable>` — Virtualized, sortable, filterable, bulk-selectable
- `<KanbanBoard>` — DnD columns with card customization
- `<Timeline>` — Activity feed with event icons
- `<StatCard>` — Metric card with trend indicator
- `<ActivityFeed>` — Real-time scrollable feed

Inputs & Forms

- `<CommandPalette>` — Global search + actions (⌘K)
- `<RichTextEditor>` — Tiptap-based with mentions
- `<FileUpload>` — Drag-drop with progress, preview
- `<DateRangePicker>` — Range selection with presets
- `<ComboBox>` — Searchable, async-loadable select

Feedback

- `<Toast>` — Position-aware notifications
- `<EmptyState>` — Contextual with CTA
- `<SkeletonLoader>` — Shimmer per component type
- `<ProgressRing>` — Animated circular progress
- `<Badge>` — Status, priority, category variants

6. Module 1 — IMS: Intern Management System

6.1 Intern Lifecycle Pipeline

Pipeline Stages (ordered, non-skippable by default):

Rationale for changes from original spec:

- Added **Screened** stage — HR reviews resume before scheduling interview. Removes noise from interview calendar.
- Added **Decision Pending** — Realistic enterprise step where panel decision takes time.
- Removed **Shortlisted** (merged into **Screened**) — redundant naming.
- Split **Completion** into **Completed** and **Alumni** — alumni is a post-internship state with different data access.

Core Features:

- Kanban view with drag-and-drop stage transitions
- Automated stage-change notifications (intern + HR)
- Stage transition rules: admin can require fields before advancing
- Stage history with actor, timestamp, and notes
- Bulk status updates with audit log
- Custom fields per stage (configurable by Super Admin)

6.2 Intern Profile

Each intern record contains:

Identity Block

- Full name, photo, DOB, gender
- Contact: email, phone, emergency contact
- Location, college/university, department, degree year

Program Block

- Domain (engineering, design, marketing, etc.)
- Mentor assigned
- Start date, expected end date, actual end date
- Internship type (remote, hybrid, on-site)
- Stipend amount and payment schedule

Status Block

- Current pipeline stage
- PPO candidate flag

- Performance tier (auto-calculated: A/B/C)
- Active/inactive flag

Access Block

- Portal credentials
- Assigned permissions
- Last login

6.3 Intern Dashboard (Intern-facing)

Clean, single-page intern home screen.

Widgets:

- Today's attendance status (with clock-in button)
- Active tasks and their deadline proximity
- Learning progress ring
- Upcoming deadlines (next 7 days)
- Recent feedback from mentor
- Performance scores (latest review cycle)
- Notifications tray

Design note: Intern dashboard should feel distinct from HR dashboard — lighter information density, more motivational tone.

6.4 Task & Productivity Management

Task Fields:

- Title, description (rich text), attachments
- Assigned to (intern), assigned by (mentor/HR)
- Priority: Critical / High / Medium / Low
- Due date, estimated hours, actual hours
- Tags and category
- Sprint association
- Approval status: Draft → Submitted → Under Review → Approved / Revision Requested

Sprint System:

- Sprints are 1-4 week cycles (configurable)
- Tasks are pulled into a sprint backlog
- Sprint velocity tracked per intern
- Sprint retrospective form (mentor fills)

Approval Workflow:

```
graph TD; A[Intern submits task] --> B[Mentor notified (real-time)]; B --> C[Mentor reviews → Approves or requests revision]; C --> D[If revision → intern gets feedback and re-submits]; D --> E[Approved → contributes to performance score];
```

Removed from original: Separate milestones tracking removed and consolidated into tasks with milestone-type flag. Reduces UI complexity.

6.5 Attendance System

Three Check-In Methods (configurable per company):

Method A: QR Code

- Admin generates rotating QR (expires every 60 seconds)
- Intern scans via mobile → attendance recorded with timestamp
- Prevents screenshot-based fraud (time-bound token)

Method B: IP-Based (replaces Wi-Fi)

- Intern must be on office network (IP range whitelisted by admin)
- Simpler than Wi-Fi SSID detection, works cross-device
- Fallback: VPN-based for remote days

Method C: Geofence

- Admin sets location + radius (e.g., 200m around office)
- GPS coordinates matched server-side
- Privacy note: location only captured at check-in/out, never tracked continuously

Attendance Record:

- Date, check-in time, check-out time
- Method used, IP address, location coordinates
- Session duration (auto-calculated)
- Day type: Working / Holiday / Leave / Half-day
- Late arrival flag (configurable threshold)

Leave Management (added — missing from original spec):

- Leave types: Casual, Sick, Compensatory, LOP
- Leave request → mentor approval workflow
- Leave balance tracker
- Calendar view with team availability

Reports:

- Monthly attendance summary per intern
- Attendance rate (%) across cohort
- Late arrival frequency
- Absent-without-leave alerts

6.6 Learning Management System (LMS)

Content Structure:

```

Program
├── Course
│   ├── Module
│   │   └── Lesson (video / article / quiz / assignment)

```

Content Types:

- Video lessons (embed YouTube/Vimeo or self-hosted)
- Articles (rich text with code blocks)
- Quizzes (MCQ, true/false, short answer)
- Coding assessments (problem statement + submission link or in-browser code runner)
- File-based assignments (upload submissions)

Progress Tracking:

- Lesson completion state (watched/read %)
- Quiz scores with attempt history
- Assignment grades with mentor feedback
- Course progress ring per intern
- Certificate auto-generated on course completion

Removed from original: "Coding assessments" with an in-browser runner is Phase 2 scope (complex to build well). Phase 1 supports problem description + external submission link (e.g., HackerRank, LeetCode).

Mentor Controls:

- Assign specific courses to interns
- Set deadlines per course

- View completion status across cohort
- Grade assignments with rubric

6.7 Performance Analytics

Score Model (4 dimensions):

Dimension	Weight	Source
Productivity	40%	Tasks: completion rate, on-time %, approval rate
Consistency	25%	Attendance rate, check-in regularity
Learning	20%	LMS progress, quiz scores
Engagement	15%	Mentor feedback, sprint participation

Overall Performance Tier:

- A (85-100): High performer, PPO candidate
- B (65-84): On track
- C (below 65): Needs attention, triggers mentor alert

AI-Assisted Insights (see AI module):

- PPO prediction score with confidence level
- Burnout risk detection (based on attendance dips + task overload)
- Skill gap analysis per intern
- Comparative peer ranking (anonymized)

Review Cycles:

- Mid-term review (auto-triggered at 50% of internship duration)
- Final review (2 weeks before end date)
- Ad-hoc review (mentor-initiated)
- Each review: structured form + free-text + rating per dimension

6.8 Document Automation

Templates managed in admin panel. Variables injected at generation time.

Documents generated:

- Offer Letter (on stage: Offer Sent)
- NDA (on stage: Onboarding)
- Experience Certificate (on stage: Completed)

- Letter of Recommendation (mentor-initiated, requires review)
- Performance Report (per review cycle)
- Internship Completion Report (batch for HR)

Format: PDF (Puppeteer-rendered, pixel-accurate). All docs stored in intern's profile document center with version history.

Removed from original: "LOR generation by AI" deferred to Phase 2. LORs require human oversight and should not be auto-published.

6.9 Communication System

Internal Channels:

- Announcement board (admin → all interns, read-only)
- Mentor ↔ Intern DM
- Team channel per domain/cohort
- Task-level comments (threaded, mention-supported)

Notifications:

- In-app (bell icon, notification center)
- Email (Resend) — daily digest + critical alerts
- WhatsApp (Phase 2, Twilio)

Removed from original: Full standalone chat system removed. Phase 1 uses task-level comments + announcements. Real-time DM (Socket.io) is Phase 2. This avoids competing with Slack and keeps scope manageable.

7. Module 2 — CMM: Client Management Module

7.1 Client Database

Client Record Schema:

Company Block:

- Company name, logo upload
- Industry vertical (dropdown, configurable)
- Company size (1-10, 11-50, 51-200, 201+)
- Website, HQ location, LinkedIn URL

Primary Contact Block:

- Name, designation, email, phone
- WhatsApp number (separate from phone)

- Communication preference (email/call/WhatsApp)

Business Block:

- Service interested (multi-select from service catalog)
- Deal value (estimated, confirmed)
- Lead source (referral, LinkedIn, cold outreach, website, etc.)
- Account owner (assigned manager)
- Secondary contacts (additional stakeholders)

Metadata:

- Created date, created by
- Last activity date (auto-updated)
- Next follow-up date (auto-updated from follow-up system)
- Tags (free-form)
- Internal rating (1-5 stars, account team only)

7.2 Client Pipeline (Sales/Delivery CRM)

Pipeline Stages:

Lead Captured → Contacted → Discovery Call Done
→ Proposal Sent → Negotiation → Closed Won / Closed Lost / On Hold

Rationale for changes:

- Renamed original stages to standard CRM terminology that aligns with B2B workflows
- Added **Discovery Call Done** — critical stage to track pre-proposal qualification
- Added **Negotiation** — common in B2B, often skipped in lightweight CRMs leading to lost deal insight
- Added **On Hold** — clients don't disappear, they pause. This prevents incorrectly closing deals.
- Removed **Need to Talk** / **Just Talked** — too informal; replaced with structured stages

Features:

- Kanban board with deal card showing company, value, owner, last activity
- Deal card aging indicator (highlight cards inactive for 7+ days)
- Stage-level conversion analytics
- Bulk reassignment (if manager leaves)
- Pipeline value sum per stage
- Won/Lost reason capture (required on closure)

7.3 Activity Timeline

Every client record has a chronological activity log.

Auto-logged activities:

- Stage changes (with actor)
- Comments added
- Documents uploaded
- Follow-up created/completed
- Invoice generated/paid

Manually logged activities:

- Outbound call (duration, summary, outcome)
- Meeting (type, attendees, notes, action items)
- WhatsApp conversation (summary)
- Email sent (subject, summary — not full content for privacy)

Each activity entry:

- Timestamp, actor, activity type, content/notes
- Pinnable (pinned activities appear at top)
- Editable within 24 hours

7.4 Comments & Internal Notes

- Threaded comments per client record
- @mention teammates (triggers notification)
- Mark comment as internal note (hidden from any future client portal)
- Pin important comments
- Emoji reactions (lightweight signal, avoids reply noise)
- Comment editing with edit history

7.5 Document Center (per client)

Categories:

- Proposals
- Contracts / Agreements
- Quotations
- Presentations
- Invoices
- Miscellaneous

Features:

- Upload (drag-drop, max 50MB per file)
- In-browser preview (PDF, images, DOCX via Google Docs viewer)
- Version history (v1, v2 per document)
- Download with access log
- Share link with expiry (for sending to client — Phase 2)
- Document status: Draft / Sent / Signed / Archived

7.6 Follow-Up Management

The most-used feature in a CRM. Must be fast and friction-free.

Follow-Up Record:

- Due date and time
- Type: Call / Email / Meeting / WhatsApp
- Priority: Urgent / Normal / Low
- Assigned to (defaults to account owner)
- Notes / talking points
- Status: Pending / Completed / Snoozed / Overdue

Daily Agenda View:

- Sorted list of today's follow-ups
- One-click mark as done
- One-click reschedule (snooze to tomorrow, next week, custom date)
- Overdue follow-ups prominently flagged

Smart Reminders:

- In-app notification 30 min before
- Email reminder day-before (configurable)
- Overdue alert to manager (if overdue > 48 hours, escalate)

Removed from original: "Meeting scheduler" (like Calendly) removed from Phase 1. Out of scope for v1; integrate with Google Calendar in Phase 2 instead.

7.7 Communication Integration

Phase 1:

- WhatsApp deep link (`(wa.me/{number}?text={template})`) — one-click opens WhatsApp with pre-filled message
- Email templates (pre-written templates per stage, variable injection)
- Email sent via Resend with delivery status tracking

Phase 2:

- Twilio WhatsApp Business API (send/receive within platform)
- Gmail / Outlook OAuth integration (send email from within platform, logged to timeline)
- Meeting scheduling via Google Calendar API

7.8 Billing & Invoices

Invoice Fields:

- Invoice number (auto-generated: INV-2024-0001)
- Bill-to (linked client)
- Line items (description, quantity, rate, tax %)
- Subtotal, tax total, grand total
- Currency (INR default, USD/EUR configurable)
- Due date
- Payment terms (Net 15, Net 30, custom)
- Notes / terms and conditions
- Status: Draft / Sent / Partially Paid / Paid / Overdue / Cancelled

Features:

- PDF generation (branded, Puppeteer)
- Send via email directly from platform (logged to timeline)
- Payment status tracking (manual mark or Razorpay/Stripe webhook in Phase 2)
- Invoice linked to client record and pipeline deal
- Revenue reports by month/quarter

Removed from original: Payment gateway integration moved to Phase 2 (Razorpay for INR, Stripe for USD). Phase 1 is manual status tracking.

8. Shared Platform Features

8.1 AI Assistant (Floating Widget)

A context-aware floating assistant — not a general chatbot.

Triggered by: Floating button (bottom-right) or  + 

Capabilities (context-aware based on current page):

Context	AI Can Do
Client record	Summarize recent activity, draft follow-up email, suggest next action
Intern profile	Summarize performance, flag risks, draft review comment
Task page	Suggest task breakdown, estimate effort
Analytics	Explain trends, compare with previous period
Empty pipeline	Suggest re-engagement strategies

Implementation: Claude API (claude-sonnet-4-20250514) with page context injected as system prompt. Streaming responses for natural feel.

Guardrails:

- Never auto-sends anything — all drafts require human review
- Responses capped at 500 tokens per turn
- Conversation resets on page navigation
- No PII sent to AI (data anonymized before sending)

8.2 Analytics Dashboard

IMS Analytics:

Metric	Visualization
Intern pipeline volume by stage	Funnel chart
Attendance rate (current cohort)	Line chart, weekly
Task completion rate by intern	Bar chart
Learning progress (cohort average)	Progress bars
Performance tier distribution	Donut chart
PPO prediction candidates	Ranked list with scores

CMM Analytics:

Global:

- Date range picker with presets (this week, month, quarter, custom)
- Export to CSV / Excel
- Scheduled reports (email weekly summary to admins)

8.3 Global Search & Command Palette

Keyboard shortcut: ⌘ K (Mac) / Ctrl K (Windows)

Searchable entities:

- Interns (name, email, domain)
- Clients (company name, contact name)
- Tasks (title)
- Documents (filename)
- Invoices (invoice number)

Command actions:

- "New client", "New intern", "New task", "New invoice"
- "Go to [page name]"
- "Change theme", "Open settings"
- Recently visited records

Implementation: Client-side search for cached data (Fuse.js), server-side search for full-text (PostgreSQL FTS) with debounce.

8.4 Notifications System

Notification types:

- **Transactional** — task assigned, stage changed, invoice due
- **Alerts** — attendance anomaly, overdue follow-up, PPO risk flagged
- **Announcements** — admin-broadcast messages
- **Digest** — daily/weekly summary (email)

Delivery channels:

- In-app notification center (bell icon, grouped by type)
- Email (configurable per notification type in user preferences)
- Real-time toast (for immediate, time-sensitive events)

Notification record retention: 90 days. Archived beyond that.

8.5 Real-Time System

Using Socket.io with Redis adapter (multi-instance safe):

Events broadcasted in real-time:

- New comment added to record you follow
- Task status changed
- Pipeline stage changed
- New announcement published
- Invoice status updated
- Follow-up reminder fires

Rooms strategy:

- `user:{userId}` — personal events
- `org:{orgId}` — org-wide announcements
- `record:{type}:{id}` — record-level events (client, intern, task)

8.6 Mobile Responsiveness

Priority tier:

- **Critical (must work on mobile):** Attendance check-in, intern dashboard, task list, notification center, daily follow-up agenda
- **Important (responsive but not primary):** Client record view, comments, activity timeline
- **Desktop-first (not mobile-optimized in Phase 1):** Analytics, document editor, admin settings, bulk operations

Native mobile app (React Native) is Phase 3.

9. Database Schema — Final Architecture

Core Principles

- **UUIDs** for all primary keys (not auto-increment integers — avoids enumeration attacks and enables distributed ID generation)
- **soft deletes** on all major entities ((deleted_at timestamp)
- **audit columns** on all tables ((created_at, updated_at, created_by, updated_by))
- **multi-tenancy** via (org_id) on every table (enables future SaaS model)

IMS Schema

sql

```

-- Organizations
organizations (
  id UUID PK,
  name VARCHAR(200),
  slug VARCHAR(100) UNIQUE,
  logo_url TEXT,
  plan ENUM('starter','pro','enterprise'),
  settings JSONB,
  created_at TIMESTAMPTZ
)

-- Users (all actors: HR, mentors, interns, admins)
users (
  id UUID PK,
  org_id UUID FK → organizations,
  email VARCHAR(255) UNIQUE,
  name VARCHAR(200),
  avatar_url TEXT,
  role ENUM('super_admin','hr_admin','mentor','client_manager','intern'),
  is_active BOOLEAN DEFAULT true,
  last_login_at TIMESTAMPTZ,
  created_at TIMESTAMPTZ,
  deleted_at TIMESTAMPTZ
)

-- Interns (extends users with intern-specific data)
interns (
  id UUID PK,
  user_id UUID FK → users,
  org_id UUID FK → organizations,
  college VARCHAR(200),
  department VARCHAR(100),
  degree_year INT,
  domain VARCHAR(100),
  mentor_id UUID FK → users,
  start_date DATE,
  expected_end_date DATE,
  actual_end_date DATE,
  stipend_amount DECIMAL(10,2),
  internship_type ENUM('remote','hybrid','on_site'),
  pipeline_stage ENUM('applied','screened','interview_scheduled','interview_done',
                      'decision_pending','offer_sent','offer_accepted','offer_reject',
                      'onboarding','active','performance_review','completed','alumni'),
  is_ppo_candidate BOOLEAN DEFAULT false,
  performance_tier CHAR(1), -- A, B, C (auto-calculated)
  overall_score DECIMAL(5,2),

```

```

    created_at TIMESTAMPTZ,
    updated_at TIMESTAMPTZ
)

-- Pipeline history (immutable log)
intern_stage_history (
    id UUID PK,
    intern_id UUID FK → interns,
    from_stage VARCHAR(50),
    to_stage VARCHAR(50),
    changed_by UUID FK → users,
    notes TEXT,
    created_at TIMESTAMPTZ
)

-- Attendance
attendance_records (
    id UUID PK,
    intern_id UUID FK → interns,
    org_id UUID FK → organizations,
    date DATE,
    check_in_at TIMESTAMPTZ,
    check_out_at TIMESTAMPTZ,
    method ENUM('qr', 'ip', 'geofence', 'manual'),
    ip_address INET,
    coordinates POINT,
    session_minutes INT GENERATED ALWAYS AS (
        EXTRACT(EPOCH FROM (check_out_at - check_in_at)) / 60
    ) STORED,
    is_late BOOLEAN DEFAULT false,
    day_type ENUM('working', 'holiday', 'leave', 'half_day'),
    notes TEXT,
    UNIQUE(intern_id, date)
)

-- Leave requests
leave_requests (
    id UUID PK,
    intern_id UUID FK → interns,
    type ENUM('casual', 'sick', 'compensatory', 'lop'),
    start_date DATE,
    end_date DATE,
    reason TEXT,
    status ENUM('pending', 'approved', 'rejected'),
    approved_by UUID FK → users,
    created_at TIMESTAMPTZ
)

```


-- Tasks

```
tasks (  
  id UUID PK,  
  org_id UUID FK → organizations,  
  title VARCHAR(500),  
  description JSONB, -- rich text as JSON (Tiptap/ProseMirror)  
  assignee_id UUID FK → users,  
  assigned_by UUID FK → users,  
  priority ENUM('critical', 'high', 'medium', 'low'),  
  status ENUM('backlog', 'in_progress', 'submitted', 'under_review', 'approved', 'revision'),  
  sprint_id UUID FK → sprints,  
  due_date DATE,  
  estimated_hours DECIMAL(5,1),  
  actual_hours DECIMAL(5,1),  
  is_milestone BOOLEAN DEFAULT false,  
  tags VARCHAR(50)[],  
  created_at TIMESTAMPTZ,  
  updated_at TIMESTAMPTZ  
)
```

-- Sprints

```
sprints (  
  id UUID PK,  
  org_id UUID FK → organizations,  
  name VARCHAR(100),  
  start_date DATE,  
  end_date DATE,  
  status ENUM('planning', 'active', 'completed'),  
  velocity_planned INT,  
  velocity_actual INT,  
  created_at TIMESTAMPTZ  
)
```

-- LMS: Courses

```
courses (  
  id UUID PK,  
  org_id UUID FK → organizations,  
  title VARCHAR(200),  
  description TEXT,  
  thumbnail_url TEXT,  
  domain VARCHAR(100),  
  is_published BOOLEAN DEFAULT false,  
  created_by UUID FK → users,  
  created_at TIMESTAMPTZ  
)
```

-- LMS: Lessons

```
lessons (  
  id UUID PK,  
  course_id UUID FK → courses,  
  module_id UUID FK → course_modules,  
  title VARCHAR(200),  
  type ENUM('video','article','quiz','assignment','coding'),  
  content JSONB,  
  order_index INT,  
  duration_minutes INT,  
  created_at TIMESTAMPTZ  
)
```

-- LMS: Enrollment & Progress

```
enrollments (  
  id UUID PK,  
  intern_id UUID FK → interns,  
  course_id UUID FK → courses,  
  assigned_by UUID FK → users,  
  due_date DATE,  
  completed_at TIMESTAMPTZ,  
  certificate_url TEXT  
)
```

```
lesson_progress (  
  id UUID PK,  
  enrollment_id UUID FK → enrollments,  
  lesson_id UUID FK → lessons,  
  status ENUM('not_started','in_progress','completed'),  
  score DECIMAL(5,2), -- for quizzes  
  completed_at TIMESTAMPTZ  
)
```

-- Performance reviews

```
performance_reviews (  
  id UUID PK,  
  intern_id UUID FK → interns,  
  reviewer_id UUID FK → users,  
  review_type ENUM('midterm','final','adhoc'),  
  productivity_score DECIMAL(5,2),  
  consistency_score DECIMAL(5,2),  
  learning_score DECIMAL(5,2),  
  engagement_score DECIMAL(5,2),  
  overall_score DECIMAL(5,2),  
  strengths TEXT,  
  improvement_areas TEXT,  
  mentor_notes TEXT,
```

```
status ENUM('draft','submitted','acknowledged'),
reviewed_at TIMESTAMPTZ,
created_at TIMESTAMPTZ
)
```

CMM Schema

sql

```

-- Clients
clients (
  id UUID PK,
  org_id UUID FK → organizations,
  company_name VARCHAR(200),
  company_logo_url TEXT,
  industry VARCHAR(100),
  company_size ENUM('1-10', '11-50', '51-200', '201+'),
  website TEXT,
  linkedin_url TEXT,
  hq_location VARCHAR(200),

  -- Primary contact
  contact_name VARCHAR(200),
  contact_designation VARCHAR(100),
  contact_email VARCHAR(255),
  contact_phone VARCHAR(20),
  contact_whatsapp VARCHAR(20),
  comm_preference ENUM('email', 'phone', 'whatsapp'),

  -- Business
  services_interested VARCHAR(100)[],
  deal_value_estimated DECIMAL(12,2),
  deal_value_confirmed DECIMAL(12,2),
  lead_source VARCHAR(100),
  account_owner_id UUID FK → users,

  -- Pipeline
  pipeline_stage ENUM('lead_captured', 'contacted', 'discovery_call_done',
                      'proposal_sent', 'negotiation', 'closed_won', 'closed_lost', 'on_
closed_reason TEXT,
closed_at TIMESTAMPTZ,

  -- Meta
  rating SMALLINT CHECK(rating BETWEEN 1 AND 5),
  tags VARCHAR(50)[],
  last_activity_at TIMESTAMPTZ,
  next_followup_date DATE,
  custom_fields JSONB,
  created_by UUID FK → users,
  created_at TIMESTAMPTZ,
  updated_at TIMESTAMPTZ,
  deleted_at TIMESTAMPTZ
)

-- Additional contacts per client

```

```
client_contacts (  
    id UUID PK,  
    client_id UUID FK → clients,  
    name VARCHAR(200),  
    designation VARCHAR(100),  
    email VARCHAR(255),  
    phone VARCHAR(20),  
    is_primary BOOLEAN DEFAULT false,  
    notes TEXT  
)  
  
-- Client pipeline stage history  
client_stage_history (  
    id UUID PK,  
    client_id UUID FK → clients,  
    from_stage VARCHAR(50),  
    to_stage VARCHAR(50),  
    changed_by UUID FK → users,  
    notes TEXT,  
    created_at TIMESTAMPTZ  
)  
  
-- Activities (unified timeline)  
client_activities (  
    id UUID PK,  
    client_id UUID FK → clients,  
    org_id UUID FK → organizations,  
    actor_id UUID FK → users,  
    type ENUM('call', 'meeting', 'whatsapp', 'email', 'note', 'stage_change',  
              'document_uploaded', 'followup_created', 'followup_completed',  
              'invoice_generated', 'invoice_paid', 'comment_added'),  
    title VARCHAR(300),  
    body TEXT,  
    metadata JSONB, -- type-specific data (duration for calls, attendees for meetings)  
    is_pinned BOOLEAN DEFAULT false,  
    created_at TIMESTAMPTZ,  
    updated_at TIMESTAMPTZ  
)  
  
-- Follow-ups  
followups (  
    id UUID PK,  
    client_id UUID FK → clients,  
    org_id UUID FK → organizations,  
    assigned_to UUID FK → users,  
    type ENUM('call', 'email', 'meeting', 'whatsapp'),  
    priority ENUM('urgent', 'normal', 'low'),
```

```

    due_at TIMESTAMPTZ,
    notes TEXT,
    status ENUM('pending', 'completed', 'snoozed', 'overdue'),
    completed_at TIMESTAMPTZ,
    snoozed_until TIMESTAMPTZ,
    created_by UUID FK → users,
    created_at TIMESTAMPTZ
)

-- Documents
client_documents (
    id UUID PK,
    client_id UUID FK → clients,
    org_id UUID FK → organizations,
    category ENUM('proposal', 'contract', 'quotation', 'presentation', 'invoice', 'misc'),
    title VARCHAR(200),
    file_url TEXT,
    file_size_bytes INT,
    mime_type VARCHAR(100),
    version INT DEFAULT 1,
    status ENUM('draft', 'sent', 'signed', 'archived'),
    uploaded_by UUID FK → users,
    created_at TIMESTAMPTZ
)

-- Invoices
invoices (
    id UUID PK,
    client_id UUID FK → clients,
    org_id UUID FK → organizations,
    invoice_number VARCHAR(50) UNIQUE, -- INV-2024-0001
    line_items JSONB, -- [{description, qty, rate, tax_pct}]
    subtotal DECIMAL(12,2),
    tax_total DECIMAL(12,2),
    grand_total DECIMAL(12,2),
    currency CHAR(3) DEFAULT 'INR',
    due_date DATE,
    payment_terms VARCHAR(50),
    notes TEXT,
    status ENUM('draft', 'sent', 'partially_paid', 'paid', 'overdue', 'cancelled'),
    sent_at TIMESTAMPTZ,
    paid_at TIMESTAMPTZ,
    pdf_url TEXT,
    created_by UUID FK → users,
    created_at TIMESTAMPTZ,

```

```
updated_at TIMESTAMPTZ
)
```

Shared Schema

```
sql
```

```

-- Comments (polymorphic – works for interns, tasks, clients)
comments (
  id UUID PK,
  org_id UUID FK → organizations,
  entity_type ENUM('intern','task','client','invoice'),
  entity_id UUID,
  parent_id UUID FK → comments, -- for threads
  author_id UUID FK → users,
  body JSONB, -- rich text
  is_internal BOOLEAN DEFAULT false,
  is_pinned BOOLEAN DEFAULT false,
  created_at TIMESTAMPTZ,
  updated_at TIMESTAMPTZ,
  deleted_at TIMESTAMPTZ
)

-- Notifications
notifications (
  id UUID PK,
  org_id UUID FK → organizations,
  recipient_id UUID FK → users,
  type VARCHAR(100), -- 'task.assigned', 'client.stage_changed', etc.
  title VARCHAR(300),
  body TEXT,
  entity_type VARCHAR(50),
  entity_id UUID,
  is_read BOOLEAN DEFAULT false,
  created_at TIMESTAMPTZ
)

-- Audit logs (immutable)
audit_logs (
  id UUID PK,
  org_id UUID FK → organizations,
  actor_id UUID FK → users,
  action VARCHAR(100), -- 'client.update', 'intern.stage_change'
  entity_type VARCHAR(50),
  entity_id UUID,
  changes JSONB, -- {field: {from, to}}
  ip_address INET,
  user_agent TEXT,
  created_at TIMESTAMPTZ
)

-- File uploads
file_uploads (

```



```
id UUID PK,  
org_id UUID FK → organizations,  
uploaded_by UUID FK → users,  
entity_type VARCHAR(50),  
entity_id UUID,  
bucket VARCHAR(100),  
key TEXT UNIQUE,  
original_name VARCHAR(255),  
mime_type VARCHAR(100),  
size_bytes BIGINT,  
is_public BOOLEAN DEFAULT false,  
created_at TIMESTAMPTZ  
)
```

10. API Design Standards

URL Convention

```
/api/v1/{module}/{resource}  
/api/v1/ims/interns  
/api/v1/ims/interns/:id  
/api/v1/ims/interns/:id/tasks  
/api/v1/cmm/clients  
/api/v1/cmm/clients/:id/activities  
/api/v1/cmm/clients/:id/followups  
/api/v1/shared/notifications  
/api/v1/shared/upload
```

Request/Response Standards

- **Pagination:** Cursor-based (not offset) for activity feeds and large lists. Offset pagination for paginated tables.
- **Filtering:** Query params — `?status=active&sort=created_at:desc&limit=20`
- **Partial updates:** `PATCH` (not `PUT`) for all updates
- **Envelope response:**

json

```
{
  "success": true,
  "data": {...},
  "meta": {
    "page": 1,
    "limit": 20,
    "total": 142,
    "cursor": "eyJpZCI6..."
  }
}
```

- **Error response:**

```
json
{
  "success": false,
  "error": {
    "code": "INTERN_NOT_FOUND",
    "message": "Intern with id xyz does not exist",
    "details": []
  }
}
```

Rate Limiting

- Standard users: 200 requests/minute
- API integrations: 1000 requests/minute
- Auth endpoints: 10 requests/minute (brute-force protection)
- Implemented via Redis + NestJS throttler guard

11. Security & Compliance

Authentication

- **Email/password** with bcrypt (rounds: 12)
- **Google OAuth 2.0** (primary for enterprise users)
- **JWT access tokens** (15-minute expiry) + refresh tokens (30-day, stored in httpOnly cookie)
- **Session management:** Redis-backed, invalidated on logout or password change

Authorization

- All API routes protected by `JwtAuthGuard`

- RBAC enforced at service layer via `PermissionsGuard` (not just controller)
- Row-level security: interns can only access their own data; mentors see their mentees only

Data Security

- All data encrypted at rest (AES-256 via database/storage layer)
- TLS 1.3 in transit (Cloudflare)
- PII columns: email, phone, coordinates — logged separately, limited access
- File uploads: scanned for MIME type mismatch, size limits enforced server-side

Audit & Compliance

- Immutable audit log for all data mutations
- Log includes: actor, timestamp, changed fields (before/after), IP, user agent
- Log retention: 2 years minimum
- Data export (DPDP compliance — India) available on request

Attack Surface Reduction

- SQL injection: impossible via Prisma parameterized queries
- XSS: Content Security Policy headers, HTML sanitized on server
- CSRF: SameSite=Strict cookies
- File upload: server-side MIME validation, stored in private S3 bucket (pre-signed URLs only)
- Enumeration: UUID PKs, generic error messages for 404s

12. Role-Based Access Control

Role Definitions

Role	Scope	Key Permissions
Super Admin	Org-wide	Full access, settings, billing, user management, audit logs
HR Admin	IMS full	Manage interns, approvals, pipeline, documents, reviews
Client Manager	CMM full	Manage clients, pipeline, invoices, follow-ups
Mentor	IMS limited	View/manage own mentees only, grade tasks, add review comments
Intern	Self only	Own dashboard, tasks, attendance, LMS, documents
Viewer	Read-only	Analytics and reports, no mutations

Permission Matrix (abbreviated)

Resource	Super Admin	HR Admin	Client Manager	Mentor	Intern
Intern list	✔ All	✔ All	✘	✔ Own mentees	✘
Intern profile	✔ R/W	✔ R/W	✘	✔ Read	✔ Own, read
Task assign	✔	✔	✘	✔ Own mentees	✘
Task submit	✔	✔	✘	✘	✔ Own tasks
Attendance	✔	✔ R/W	✘	✔ Read	✔ Own, mark
Client list	✔	✘	✔ All	✘	✘
Invoices	✔	✘	✔	✘	✘
Analytics	✔	✔ IMS only	✔ CMM only	✔ Limited	✘
Settings	✔	✘	✘	✘	✘
Audit logs	✔	✘	✘	✘	✘

13. Automation Engine

Rule Format

Rules are stored in the database as JSON and evaluated by a BullMQ job processor.

```
typescript
interface AutomationRule {
  id: string;
  orgId: string;
  name: string;
  trigger: {
    event: 'task.overdue' | 'intern.stage_changed' | 'client.inactive' |
           'attendance.missed' | 'review.due' | 'followup.overdue' | 'invoice.overdue';
    conditions: Condition[];
  };
  actions: Action[];
  isActive: boolean;
}
```

Built-in Automations (pre-configured, toggleable)

Trigger	Condition	Action
Task overdue	due_date < now AND status != done	Notify assignee + mentor
Attendance missed	3 consecutive absences	Alert HR Admin + mentor
Intern stage → Offer Sent	—	Generate offer letter PDF, send to intern
Intern stage → Completed	—	Generate certificate, send LOR prompt to mentor
Intern performance_tier = C	consecutive 2 reviews	Flag for mentor intervention, notify HR
Client inactive	last_activity_at > 14 days	Notify account owner
Follow-up overdue	due_at < now - 48h	Escalate to manager
Invoice overdue	due_date < now AND status = sent	Send payment reminder email

Custom Automation Builder (Phase 2)

Visual if/then builder for non-technical admins. Phase 1 provides the pre-built rules above via toggle switches.

14. AI Integration Layer

Architecture



AI Features by Priority

Phase 1 (ship with v1):

- AI-powered email/message drafting (provide context, generate draft)
- Performance review summary generation (from scores + notes)
- Follow-up suggestion after activity (call → suggest next action)
- Anomaly explanation (attendance drop — explain in plain English)

Phase 2:

- PPO candidate scoring model (regression model, trained on historical data)
- Burnout risk detection (trend analysis on attendance + task velocity)
- Client sentiment analysis (activity patterns → churn risk)
- Smart task estimation (based on similar past tasks)

Phase 3:

- RAG-based knowledge assistant (answer questions from uploaded company docs)
- AI-driven pipeline coaching (suggest actions per deal stage)

Context Injection Strategy

Never send raw PII to the AI. Always anonymize or pseudonymize:

```
typescript

// Bad
const prompt = `Client: Ramesh Kumar, +91-9876543210, email: ramesh@company.com...`

// Good
const prompt = `Client company is in SaaS industry, deal value ~5L INR,
last contacted 12 days ago, stage: proposal_sent, no activity since.
Draft a follow-up email subject and body.`
```

15. Infrastructure & DevOps

Environments

Environment	Purpose	Hosting
Development	Local development	Docker Compose
Staging	QA, testing, demos	Railway / Render
Production	Live users	AWS ECS (Fargate) + RDS

CI/CD Pipeline (GitHub Actions)

Push to feature branch

↓

Lint (ESLint) + Type check (tsc --noEmit)

↓

Unit tests (Jest)

↓

Build check (next build + nest build)

↓

PR opened → Deploy preview (Vercel for frontend)

↓

Merge to main → Deploy to staging

↓

Manual approval → Deploy to production

Monitoring & Observability

- **Error tracking:** Sentry (frontend + backend)
- **Product analytics:** PostHog (feature usage, funnel analysis)
- **Infrastructure metrics:** AWS CloudWatch
- **Log aggregation:** Logtail / Axiom
- **Uptime monitoring:** Better Uptime (99.9% SLA target)
- **Performance:** Web Vitals tracked via Next.js Analytics

Backup & Disaster Recovery

- PostgreSQL: daily automated snapshots (30-day retention)
- S3: versioning enabled, cross-region replication
- Recovery Time Objective (RTO): < 4 hours
- Recovery Point Objective (RPO): < 24 hours

Caching Strategy

Layer	Technology	TTL	What's Cached
API response	Redis	5 min	Analytics, dashboard stats
Session	Redis	30 days	Auth tokens
Search index	In-memory (TanStack Query)	5 min	Search results
File CDN	Cloudflare	24 hrs	Static assets, document previews

16. Build Phases & Milestones

Phase 0 — Foundation (Weeks 1-2)

- Monorepo setup (Turborepo)
- Auth system (NextAuth, RBAC skeleton)
- Design system (component library baseline)
- Database migrations (Prisma)
- CI/CD pipeline
- Staging environment

Phase 1 — IMS Core (Weeks 3-6)

- Intern CRUD and pipeline
- Attendance system (QR + IP methods)
- Task management + approval workflow
- LMS (courses, lessons, progress)
- Performance scores (manual review)
- Document generation (offer letter, certificate)
- Intern dashboard

Phase 2 — CMM Core (Weeks 7-10)

- Client database + pipeline
- Activity timeline
- Follow-up management
- Document center
- Invoice generation + PDF
- Email integration (Resend)
- Follow-up reminders

Phase 3 — Intelligence Layer (Weeks 11-13)

- AI assistant (Claude API)
- Analytics dashboards (IMS + CMM)
- Automation engine (built-in rules)
- Global search + command palette
- Real-time notifications (Socket.io)

Phase 4 — Polish & Scale (Weeks 14-16)

- Mobile responsiveness pass

- Performance optimization (Core Web Vitals)
 - Audit logs + security hardening
 - Export system (CSV/Excel)
 - User onboarding flows
 - Load testing (k6)
 - Production deployment
-

17. Feature Audit — Removed, Retained, Upgraded

✖ Removed (with rationale)

Feature	Reason
Internet Identity (Dfinity)	ICP-specific, wrong ecosystem for a web SaaS
In-browser coding assessments	Significant scope — external tools (HackerRank) used in Phase 1
Full internal chat system	Competes with Slack; Phase 1 uses comments + announcements
WhatsApp API (native)	Requires Meta verification; replaced with deep linking in Phase 1
Google Docs-style collaborative editing	Out of scope; Tiptap is single-user rich text
Meeting scheduler (Calendly-like)	Phase 2: Google Calendar integration
Payment gateway (Razorpay/Stripe)	Phase 2: manual status tracking in Phase 1
AI-generated LORs auto-published	Requires human review gate; auto-publish is risky
WiFi SSID-based attendance	Unreliable across devices/OS; replaced with IP range check
Geo-continuous tracking	Privacy violation risk; replaced with geofence check-in/out only

✔ Retained (core features, kept as-is)

- Intern lifecycle pipeline (enhanced with better stages)
- Kanban boards (intern pipeline + client pipeline)
- Task management + approval workflow
- QR attendance
- LMS (courses, lessons, quizzes)

- Performance review system
- Document center (client + intern)
- Follow-up management with reminders
- Invoice generation (PDF)
- RBAC with all 6 roles
- Global search + command palette
- Real-time notifications
- Analytics dashboards

 Upgraded (enhanced beyond original spec)

Feature	Original	Upgraded
Attendance	Basic log	QR anti-fraud (time-bound tokens), leave management, IP range
Performance scoring	4 generic scores	Weighted model with auto-tier calculation and AI risk flags
Pipeline stages	Informal names	Standard CRM terminology with transition rules and won/lost capture
Automation engine	IF/THEN examples	Structured rule schema, BullMQ-backed, toggleable built-ins
AI assistant	Generic chatbot	Context-aware, page-scoped, PII-anonymized, streaming
Database	Table list	Full schema with UUIDs, soft deletes, multi-tenancy, audit columns
Invoice	Basic fields	Full line items, taxes, PDF generation, timeline integration
Comments	Simple notes	Polymorphic (works on any entity), threaded, @mentions, internal flag

18. Success Metrics & KPIs

Product Metrics (6-month post-launch)

Metric	Target
Daily active users (interns)	> 70% of enrolled interns log in weekly

Metric	Target
Attendance marked digitally	> 90% (vs manual)
Task approval cycle time	< 48 hours average
Follow-up completion rate	> 80% on-time
Pipeline conversion rate	Baseline captured in month 1
AI assistant usage	> 30% of managers use weekly

Technical Metrics

Metric	Target
API p99 latency	< 300ms
API p50 latency	< 80ms
Frontend LCP	< 2.5s
Uptime	> 99.5%
Test coverage (backend)	> 70%
Build time (CI)	< 4 minutes

Appendix A — Third-Party Services Summary

Service	Purpose	Tier	Notes
Anthropic Claude API	AI assistant	Pay-per-use	claude-sonnet-4-20250514
Resend	Transactional email	Free tier → Pro	High deliverability
AWS S3	File storage	Pay-per-use	Or Supabase Storage
Cloudflare	CDN + security	Free → Pro	DNS, DDoS protection
Sentry	Error tracking	Free tier → Team	Frontend + backend
PostHog	Product analytics	Free tier	Self-hostable option
BullMQ + Redis	Job queues	Included with Redis	Upstash Redis for managed
Puppeteer	PDF generation	Free (self-hosted)	Runs in Node.js on server

Appendix B — Naming Conventions

Files:	kebab-case	(intern-card.tsx, client-pipeline.ts)
Components:	PascalCase	(InternCard, ClientPipeline)
Functions:	camelCase	(getInternById, updateClientStage)
Database tables:	snake_case	(intern_stage_history, client_activities)
API routes:	kebab-case	(/api/v1/ims/stage-history)
Env variables:	SCREAMING_SNAKE	(DATABASE_URL, ANTHROPIC_API_KEY)
Git branches:	feat/fix/chore	(feat/intern-attendance-qr)

Document maintained by TechMecha Torque Engineering. Update version number on any structural change. Last updated: 2026 · Status: Approved for Development